

DOCKER GUIDE

1) Navigate to the Docker folder

Open a terminal and go to the project directory:

```
bash
```

```
cd ~/robotics_essentials_ros2/docker
```

2) Start the Docker container (MANDATORY FIRST STEP)

```
bash
```

```
docker compose up
```

This starts the ROS2 environment container.

```
=> => naming to docker.io/library/robotics_essentials_ros2 0.0s
=> [robotics_essentials_ros2] resolving provenance for metadata file 0.0s
[+] Running 1/0
✓ Container robotics_essentials_ros2 Created 0.0s
Attaching to robotics_essentials_ros2
```

⚠ Do this **before running any ROS commands**.

3) Open another terminal for working inside the container

Do **not** stop the running container terminal.

Open a new terminal window/tab.

Shortcut:

Alt + + (for split terminal)

4) Enter the running container

In the new terminal, run:

```
docker exec -it robotics_essentials_ros2 bash
```

You are now inside the container environment.

All ROS commands, builds, simulations, and code execution must be done here.

ROS 2 Guide

Step 1: Create Workspace and Package

1.1 Create workspace folder structure

```
bash
mkdir -p ~/pixelbotix_ws/src
```

1.2 Navigate to workspace

```
bash
cd ~/pixelbotix_ws/
```

1.3 Build workspace (this creates install, build, log folders)

```
bash
colcon build
```

1.4 Source workspace (add to .bashrc for permanent)

```
bash
echo "source ~/pixelbotix_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

1.5 Navigate to src folder

```
bash
cd src
```

1.6 Create package

```
bash
ros2 pkg create my_robot --build-type ament_python --dependencies rclpy std_msgs
geometry_msgs turtlesim example_interfaces
```

1.7 Go inside the package folder to see what was created

```
bash
cd my_robot
ls -la
```

Expected Output:

```
text
drwxr-xr-x 4 user user 4096 date .
drwxr-xr-x 3 user user 4096 date ..
drwxr-xr-x 2 user user 4096 date my_robot
-rw-r--r-- 1 user user 43 date package.xml
-rw-r--r-- 1 user user 837 date setup.cfg
-rw-r--r-- 1 user user 1456 date setup.py
drwxr-xr-x 3 user user 4096 date test
```

1.8 Go back to workspace and build

```
bash
cd ~/pixelbotix_ws
colcon build
```



Step 2: Create Basic Publisher Node

2.1 Navigate to package Python folder

```
bash
cd ~/pixelbotix_ws/src/my_robot/my_robot
```

2.2 Check what's inside

```
bash
ls -la
```

2.3 Create publisher file

```
bash
touch my_publisher.py
chmod +x my_publisher.py
```

2.4 Edit the file with nano

```
bash
nano my_publisher.py
```

2.5 Copy and paste this exact code:

```
python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class MyPublisher(Node):
    def __init__(self):
        super().__init__('my_publisher')

        # Create publisher
        self.publisher_ = self.create_publisher(String, 'chatter', 10)

        # Create timer for publishing
        timer_period = 1.0 # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.count = 0

    def timer_callback(self):
        # Create message
        msg = String()
        msg.data = f'Hello ROS 2: {self.count}'

        # Publish message
        self.publisher_.publish(msg)
        self.get_logger().info(f'Publishing: "{msg.data}"')
```

```
        self.count += 1

def main(args=None):
    rclpy.init(args=args)

    # Create and run publisher node
    publisher_node = MyPublisher()

    try:
        rclpy.spin(publisher_node)
    except KeyboardInterrupt:
        pass
    finally:
        # Cleanup
        publisher_node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

2.6 Save and exit

Press `Ctrl+X`, then `Y`, then `Enter`



Step 3: Create Basic Subscriber Node

3.1 Stay in the same folder

```
bash
pwd
```

Should show: `/home/your_username/pixelbotix_ws/src/my_robot/my_robot`

3.2 Create subscriber file

```
bash
touch my_subscriber.py
chmod +x my_subscriber.py
```

3.3 Edit the file

bash

```
nano my_subscriber.py
```

3.4 Copy and paste this exact code:

python

```
#!/usr/bin/env python3
```

```
import rclpy
```

```
from rclpy.node import Node
```

```
from std_msgs.msg import String
```

```
class MySubscriber(Node):
```

```
    def __init__(self):
```

```
        super().__init__('my_subscriber')
```

```
        # Create subscriber
```

```
        self.subscription = self.create_subscription(
```

```
            String,
```

```
            'chatter',
```

```
            self.listener_callback,
```

```
            10)
```

```
        self.subscription # prevent unused variable warning
```

```
    def listener_callback(self, msg):
```

```
        self.get_logger().info(f'I heard: "{msg.data}"')
```

```
def main(args=None):
```

```
    rclpy.init(args=args)
```

```
    # Create and run subscriber node
```

```
    subscriber_node = MySubscriber()
```

```
    try:
```

```
        rclpy.spin(subscriber_node)
```

```
    except KeyboardInterrupt:
```

```
        pass
```

```
    finally:
```

```
        subscriber_node.destroy_node()
```

```
        rclpy.shutdown()
```

```
if __name__ == '__main__':
```

```
    main()
```

3.5 Save and exit

Press Ctrl+X, then Y, then Enter

Step 4: Update Setup.py for Node Entry Points

4.1 Navigate to package root

```
bash
cd .. # Go back to ~/pixelbotix_ws/src/my_robot/
pwd   # Should show: /home/your_username/pixelbotix_ws/src/my_robot
```

4.2 Check current setup.py

```
bash
cat setup.py
```

4.3 Edit setup.py

```
bash
nano setup.py
```

4.4 Find the ***entry_points*** section and add publisher and subscriber:

```
Python
entry_points={
    'console_scripts':[
        'my_publisher = my_robot.my_publisher:main',
        'my_subscriber = my_robot.my_subscriber:main',
    ],
},
```

4.5 Save and exit

Press Ctrl+X, then Y, then ENTER

Step 5: Build and Test Publisher/Subscriber

5.1 Navigate to workspace and build

```
bash
cd ~/pixelbotix_ws
colcon build
```

Expected Build Output:

```
text
Starting >>> my_robot
Finished <<< my_robot [X.XX seconds]

Summary: 1 package finished [X.XX seconds]
```

5.2 Source the workspace

```
bash
source install/setup.bash
```

5.3 Open Terminal 1 - Run Publisher

```
bash
ros2 run my_robot my_publisher
```

Expected Output (Terminal 1):

```
text
[INFO] [my_publisher]: Publishing: "Hello ROS 2: 0"
[INFO] [my_publisher]: Publishing: "Hello ROS 2: 1"
[INFO] [my_publisher]: Publishing: "Hello ROS 2: 2"
... (continues every second)
```

5.4 Open Terminal 2 - Run Subscriber

```
bash
cd ~/pixelbotix_ws
source install/setup.bash
ros2 run my_robot my_subscriber
```

Expected Output (Terminal 2):

```
text
[INFO] [my_subscriber]: I heard: "Hello ROS 2: 0"
[INFO] [my_subscriber]: I heard: "Hello ROS 2: 1"
[INFO] [my_subscriber]: I heard: "Hello ROS 2: 2"
... (matches publisher output)
```

5.5 Stop both with Ctrl+C

Press Ctrl + C in both terminals

Step 6: Create TurtleSim Controller

6.1 Navigate to package Python folder

```
bash
cd ~/pixelbotix_ws/src/my_robot/my_robot
```

6.2 Create turtle controller file

```
bash
touch turtle_controller.py
chmod +x turtle_controller.py
```

6.3 Edit the file

```
ash
nano turtle_controller.py
```

6.4 Copy and paste this exact code:

```
python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
```

```
class TurtleController(Node):
    def __init__(self):
        super().__init__('turtle_controller')

        # Publisher to control turtle
        self.publisher_ = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)

        # Create timer
        timer_period = 0.1 # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.start_time = self.get_clock().now()

    def timer_callback(self):
        # Create Twist message
        move_cmd = Twist()

        # Make turtle move in a circle
        move_cmd.linear.x = 2.0 # Constant forward speed
        move_cmd.angular.z = 1.0 # Constant turning speed

        # Publish command
        self.publisher_.publish(move_cmd)
        self.get_logger().info(f'Moving turtle: linear={move_cmd.linear.x}, angular=
{move_cmd.angular.z}')

def main(args=None):
    rclpy.init(args=args)

    turtle_controller = TurtleController()

    try:
        rclpy.spin(turtle_controller)
    except KeyboardInterrupt:
        pass
    finally:
        turtle_controller.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

6.5 Save and exit

Press `Ctrl+X`, then `Y`, then `Enter`

Step 7: Create Turtle Position Monitor

7.0 Navigate to package Python folder

```
bash
cd ~/pixelbotix_ws/src/my_robot/my_robot
```

7.1 Stay in same folder

```
bash
pwd # Should be in ~/pixelbotix_ws/src/my_robot/my_robot
```

7.2 Create turtle position file

```
bash
touch turtle_position.py
chmod +x turtle_position.py
```



7.3 Edit the file

```
bash
nano turtle_position.py
```

7.4 Copy and paste this exact code:

```
python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from turtlesim.msg import Pose

class TurtlePositionSubscriber(Node):
    def __init__(self):
        super().__init__('turtle_position_subscriber')
```

```
# Subscribe to turtle pose
self.subscription =
    self.create_subscription(Pose,
        '/turtle1/pose',
        self.pose_callback,
        10)

def pose_callback(self, msg):

    self.get_logger().info(f'Turtle Position: x={msg.x:.2f}, y={msg.y:.2f},
        theta={msg.theta:.2f}')

def main(args=None):
    rclpy.init(args=args)

    position_subscriber = TurtlePositionSubscriber()
    position_subscriber.get_logger().info("Listening to turtle position...")
    try:
        rclpy.spin(position_subscriber)
    except KeyboardInterrupt:
        pass
    finally:
        position_subscriber.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

7.5 Save and exit

Press Ctrl + s and Ctrl+ X then Enter

7.6 Update Setup.py

python

```
entry_points={
    'console_scripts':[
        'turtle_controller = my_robot.turtle_controller:main',
        'turtle_position = my_robot.turtle_position:main',
    ],
},
```

Step 8: Build and Test TurtleSim with Teleop Key

8.1 Navigate to workspace and build

```
bash
cd ~/pixelbotix_ws
colcon build
source install/setup.bash
```

8.2 Open 3 terminals:

Terminal 1: Start TurtleSim

```
bash
ros2 run turtlesim turtlesim_node
```

Expected Output: TurtleSim window opens with turtle at center

Terminal 2: Start Turtle Position Monitor

```
bash
cd ~/pixelbotix_ws
source install/setup.bash
ros2 run my_robot turtle_position
```

Expected Output:

```
text
[INFO] [turtle_position_subscriber]: Listening to turtle position...
[INFO] [turtle_position_subscriber]: Turtle Position: x=5.54, y=5.54, theta=0.00
```

Terminal 3: Start turtle_controller

```
bash
cd ~/pixelbotix_ws
source install/setup.bash
ros2 run my_robot turtle_controller
```

STOP THE TURTLE CONTROLLER AND RUN TELEOP KEY

Terminal 3: Start Teleop Key Controller

```
bash
cd ~/pixelbotix_ws
source install/setup.bash
ros2 run turtlesim turtle_teleop_key
```



Expected Output:

text

```
[INFO] [turtle_teleop_key]: Turtle Teleop Key Controller Started!  
[INFO] [turtle_teleop_key]: Use WASD keys to control the turtle  
[INFO] [turtle_teleop_key]: W: Forward, S: Backward, A: Left, D: Right  
[INFO] [turtle_teleop_key]: Press 'Q' to quit
```

8.3 Control the Turtle with Keyboard

In Terminal 3 (where teleop is running), use these keys:

- **W** - Move forward
- **S** - Move backward
- **A** - Turn left (rotate counter-clockwise)
- **D** - Turn right (rotate clockwise)
- **Space** - Stop
- **Q** - Quit the program

Expected Output in Terminal 3 when pressing keys:

text

```
[INFO] [turtle_teleop_key]: Moving FORWARD  
[INFO] [turtle_teleop_key]: Turning LEFT  
[INFO] [turtle_teleop_key]: STOPPING
```

Expected Output in Terminal 2 (position monitor):

text

```
[INFO] [turtle_position_subscriber]: Turtle Position: x=5.54, y=5.54, theta=0.00  
[INFO] [turtle_position_subscriber]: Turtle Position: x=5.65, y=5.65, theta=0.10  
[INFO] [turtle_position_subscriber]: Turtle Position: x=5.75, y=5.75, theta=0.20
```

8.4 Stop all with Ctrl+C in each terminal

Press Ctrl + C in all the three terminals

Step 9: Create Service Server

9.1 Navigate to package Python folder

```
bash
cd ~/pixelbotix_ws/src/my_robot/my_robot
```

9.2 Create service server file

```
bash
touch add_two_ints_server.py
chmod +x add_two_ints_server.py
```

9.3 Edit the file

```
bash
nano add_two_ints_server.py
```

9.4 Copy and paste this exact code:

```
python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class AddTwoIntsServer(Node):
    def __init__(self):
        super().__init__('add_two_ints_server')

        # Create service
        self.srv = self.create_service(
            AddTwoInts,
            'add_two_ints',
            self.add_two_ints_callback
        )
        self.get_logger().info('AddTwoInts service server ready...')
```

```
def add_two_ints_callback(self, request, response):
    response.sum = request.a + request.b
    self.get_logger().info(f'Incoming request: {request.a} + {request.b} =
{response.sum}')
    return response

def main(args=None):
    rclpy.init(args=args)
    server = AddTwoIntsServer()
    try:
        rclpy.spin(server)
    except KeyboardInterrupt:
        pass
    finally:
        server.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

9.5 Save and exit

Press Ctrl+X, then Y, then Enter



Step 10 : Create CLIENT FOR THE SERVER

10.1 Navigate to package Python folder

```
bash
cd ~/pixelbotix_ws/src/my_robot/my_robot
```

10.2 Create service server file

```
bash
touch add_two_ints_client.py
chmod +x add_two_ints_client.py
```

10.3 Edit the file

```
bash
nano add_two_ints_client.py
```

10.4 Copy and paste this exact code:

```
python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class AddTwoIntsClient(Node):
    def __init__(self):
        super().__init__('add_two_ints_client')

        # Create service
        self.client = self.create_client(AddTwoInts, 'add_two_ints')

        # Wait for service to be available
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Waiting for add_two_ints service...')

        # Create request object
        self.request = AddTwoInts.Request()

    def send_request(self, a, b):

        self.request.a = a
        self.request.b = b

        self.future = self.client.call_async(self.request)
        rclpy.spin_until_future_complete(self, self.future)
        return self.future.result()

def main(args=None):
    rclpy.init(args=args)
    server = AddTwoIntsClient()
    a=5
    b=7
    client.get_logger().info(f'Result of {a} + {b} = {response.sum}')

    client.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

10.5 Save and exit

Press `Ctrl+X`, then `y`, then `Enter`

10.6 Update Setup.py

python

```
entry_points={
    'console_scripts':[
        'add_two_ints_server = my_robot.add_two_ints_server:main',
        'add_two_ints_client = my_robot.add_two_ints_client:main',
    ],
},
```

Step 11: Build and Test Service

11.1 Navigate to workspace and build

bash

```
cd ~/pixelbotix_ws
```

```
colcon build
```

```
source install/setup.bash
```

11.2 Open 2 terminals:

Terminal 1: Start Service Server

bash

```
ros2 run my_robot add_two_ints_server
```

Expected Output:

text

```
[INFO] [add_two_ints_server]: AddTwoInts service server ready...
```

Terminal 2: Call Service

bash

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 5, b: 3}"
```

Expected Output (Terminal 2):

```
text
requester: making request: example_interfaces.srv.AddTwoInts_Request(a=5, b=3)

response:
example_interfaces.srv.AddTwoInts_Response(sum=8)
```

Expected Output (Terminal 1 - Server):

```
text
[INFO] [add_two_ints_server]: Incoming request: 5 + 3 = 8
```

11.3 Stop server with Ctrl+C

Press Ctrl + c in all the terminals

Step 12: Create Parameter Example

12.1 Navigate to package Python folder

```
bash
cd ~/pixelbotix_ws/src/my_robot/my_robot
```

12.2 Create parameter example file

```
bash
touch parameter_example.py
chmod +x parameter_example.py
```

12.3 Edit the file

```
bash
nano parameter_example.py
```

12.4 Copy and paste this exact code:

```
python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class ParameterExample(Node):
    def __init__(self):
        super().__init__('parameter_example')

        # Declare parameters
        self.declare_parameter('robot_name', 'pixelbotix_robot')
        self.declare_parameter('max_velocity', 2.0)
        self.declare_parameter('enable_navigation', True)

        # Get parameters
        robot_name = self.get_parameter('robot_name').value
        max_vel = self.get_parameter('max_velocity').value
        enable_nav = self.get_parameter('enable_navigation').value

        self.get_logger().info(f'Robot Name: {robot_name}')
        self.get_logger().info(f'Max Velocity: {max_vel}')
        self.get_logger().info(f'Navigation Enabled: {enable_nav}')

        # Create timer to demonstrate parameter updates
        self.timer = self.create_timer(5.0, self.parameter_demo)

    def parameter_demo(self):
        # Demonstrate parameter description
        params = self.get_parameters(['robot_name', 'max_velocity', 'enable_navigation'])

def main(args=None):
    rclpy.init(args=args)

    param_node = ParameterExample()

    try:
        rclpy.spin(param_node)
    except KeyboardInterrupt:
        pass
    finally:
```

```
param_node.destroy_node()
rclpy.shutdown()

if __name__ == '__main__':
    main()
```

12.5 Save and exit

Press `Ctrl+X`, then `Y`, then `Enter`

12.6 Update Setup.py

```
python
entry_points={
    'console_scripts':[
        'parameter_example = my_robot.parameter_example:main',
    ],
},
```

Step 13: Build and Test Parameters

13.1 Navigate to workspace and build

```
bash
cd ~/pixelbotix_ws
colcon build
source install/setup.bash
```

13.2 Run parameter example

```
bash
ros2 run my_robot parameter_example
```

Expected Output:

```
text
[INFO] [parameter_example]: Robot Name: pixelbotix_robot
[INFO] [parameter_example]: Max Velocity: 2.0
[INFO] [parameter_example]: Navigation Enabled: True
```

```
[INFO] [parameter_example]: Current parameters:  
[INFO] [parameter_example]:   robot_name: pixelbotix_robot  
[INFO] [parameter_example]:   max_velocity: 2.0  
[INFO] [parameter_example]:   enable_navigation: True
```

```
[INFO] [parameter_example]: Current parameters:  
[INFO] [parameter_example]:   robot_name: pixelbotix_robot  
... (repeats every 5 seconds)
```

13.3 Update the param with the following commands

```
ros2 param set /parameter_example robot_name "atlas_bot"  
ros2 param set /parameter_example max_velocity 3.5  
ros2 param set /parameter_example enable_navigation false
```

13.4 Stop with Ctrl+C

Press `Ctrl+C`

Step 14 : View ROS 2 Graph

14.1 Navigate to workspace

```
bash  
cd ~/pixelbotix_ws  
source install/setup.bash
```

14.2 Open 3 terminals:

Terminal 1: Start TurtleSim

```
bash  
ros2 run turtlesim turtlesim_node
```

Terminal 2: Start turtle controller

```
bash  
ros2 run my_robot turtle_controller
```

Terminal 3: View ROS graph

```
bash
```

```
# Method 1: Text-based node List
```

```
ros2 node list
```

```
# Method 2: Text-based topic List
```

```
ros2 topic list
```

```
# Method 3: Visual graph (if rqt is installed)
```

```
rqt_graph
```

Expected Output from `ros2 node list`:

```
text
```

```
/turtle_controller
```

```
/turtlesim
```

Expected Output from `ros2 topic list`:

```
text
```

```
/chatter
```

```
/parameter_events
```

```
/rosout
```

```
/turtle1/cmd_vel
```

```
/turtle1/color_sensor
```

```
/turtle1/pose
```



RQT Graph: Shows visual connection diagram of all nodes and topics

Final Folder Structure:

text

```
pixelbotix_ws/
├── src/
│   └── my_robot/
│       ├── my_robot/
│       │   ├── my_publisher.py
│       │   ├── my_subscriber.py
│       │   ├── turtle_controller.py
│       │   ├── turtle_position.py
│       │   ├── add_two_ints_server.py
│       │   └── parameter_example.py
│       ├── package.xml
│       ├── setup.py
│       └── test/
├── build/
├── install/
└── log/
```

Quick Navigation Commands:

bash

Go to workspace

cd ~/pixelbotix_ws

Go to package

cd ~/pixelbotix_ws/src/my_robot

Go to Python code folder

cd ~/pixelbotix_ws/src/my_robot/my_robot